



Enhanced intrusion detection system using feature selection and hybrid learning models for high performance and efficiency in an IOT environment

Omar Achbarou^{a,*}, Toufik Datsi^b, Outmane Bourkougou^c, Ahmed My El kiram^d

^a Cadi Ayyad University, National School of Applied Sciences, LMSC Laboratory, Marrakech, Morocco

^b Ibn Zohr University, Computer Systems and Vision Laboratory, Agadir, Morocco

^c Cadi Ayyad University, Laboratory of computer Science and System engineering (L2IS), Marrakech, Morocco

^d Cadi Ayyad University, Information Systems Engineering Laboratory, Marrakech, Morocco

ARTICLE INFO

Keywords:

IoT
Cybersecurity
CNN
Deep learning
IDS
Feature selection

ABSTRACT

Attacks on automated systems are increasingly frequent, sophisticated, intelligent and distributed, posing serious threats to an Internet of Things (IoT) environment and necessitating costly interventions. Intrusion detection systems (IDSs) therefore play an essential role in protecting IoT environments against these types of attack. However, traditional IDS detection methods, which are mainly based on signatures and rules, face significant challenges such as an inability to detect unknown attacks, resulting in low detection rates, high false positive rates and long response times.

To overcome these limitations, this paper proposes a novel deep learning approach for binary classification of network traffic (normal vs. attack) in the context of anomaly detection, using convolutional neural networks (CNNs) and feature selection techniques for an IDS. A CNN is leveraged for its ability to detect spatial patterns in network characteristics, a crucial capability in dynamic and distributed IoT environments where the modeling of normal behavior is challenging. To mitigate data imbalance, our model applies a two-stage strategy: first, the Synthetic Minority Oversampling Technique (SMOTE) is used for global balancing before dataset splitting, and second, a class weighting technique is applied after splitting, during training. The aim of this approach is to enhance detection accuracy and reduce the execution time, to provide a more effective IDS for IoT networks.

We conduct experiments on CICIDS2017, a widely used benchmark dataset, to evaluate and compare the performance of Random Forest (RF), LightGBM and XGBoost classifiers, both with and without feature selection. The results show remarkable improvements in regard to accuracy (99.95 %), precision (99.99 %), and recall (99.91 %), especially with the XGBoost model with 40 selected features, while maintaining a very competitive runtime that is significantly lower than comparable methods in existing studies. Furthermore, our model outperforms other existing schemes in terms of performance, efficiency and scalability.

Introduction

With the rapid growth of IoT and cloud computing, their technical, economic, energy, and commercial dimensions have gained importance. However, privacy and security continue to be major obstacles to their widespread use. IoT environments are becoming more vulnerable to distributed attacks and security threats due to the growing number of users and services [1]. More complex frequent and potent cyberattacks are frequently resulting in serious data breaches monetary losses and system damage. Interventions that address these threats must be expensive and intricate.

An IDS is an important solution for minimizing the attack surface and

mitigating unauthorized access or malicious behavior within the IoT environment. However, the techniques used by current IDSs remain traditional, and mainly use signature-based and anomaly-based detection techniques [2]; these face significant challenges in regard to adapting to new attacks, meaning that they often have high false positive rates.

To overcome the limitations of these detection techniques, IDSs based on machine learning (ML) methods, such as support vector machine (SVM) and decision trees, are being deployed, meaning that extensive feature engineering and the knowledge and involvement of experts are required to process the voluminous amounts of data produced in the IoT [3]. In addition, these methods often cannot be

* Corresponding author.

E-mail address: o.achbarou@uca.ac.ma (O. Achbarou).

<https://doi.org/10.1016/j.jer.2025.10.016>

Received 13 January 2025; Received in revised form 19 October 2025; Accepted 23 October 2025

Available online 26 October 2025

2307-1877/© 2025 The Author(s). Published by Elsevier B.V. on behalf of Kuwait University. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

generalized to new types of attack, and suffer from multi-class and scalability problems when applied to large-scale datasets.

Research in the area of IDSs has advanced through the use of deep learning (DL), a sub-field of ML in which proactive measures are applied to identify and deter malicious network traffic. CNNs, in particular, offer a unique advantage, as they can identify spatial patterns and correlations in network traffic without the need for manual feature engineering [4]. This capability enables CNN-based IDS systems to detect both known and unknown intrusions with greater accuracy.

Our contribution is therefore to develop an efficient and scalable IDS based on an advanced CNN architecture combined with feature selection and dataset balancing techniques, using the CICIDS2017 dataset. The proposed system is capable of operating in real time within the IoT network, and we leverage the automatic feature extraction capabilities of deep learning. The aims of our model are to enhance detection accuracy and to reduce both the false positive rate and execution time, while ensuring the system's ability to adapt to emerging attacks in the IoT environment.

The remainder of this paper is structured as follows: Section 2 presents the background to the study, while Section 3 gives an overview of previous work. Section 4 presents our proposed model, giving a detailed overview of its functionalities. System performance tests were conducted, and the corresponding experimental results are presented in Section 5. Section 6 contains an in-depth evaluation of the performance and overall effectiveness of the proposed system, and compares our model with existing schemes in Section 7. The last section concludes this research and presents future work.

Literature review

Security in the IoT environment

In the IoT, security is required to support several important services, such as confidentiality, integrity, availability, authentication, access control and non-repudiation. These services are key aspects that IoT service providers must guarantee through a service-level agreement (SLA). This agreement is essential, since IoT systems often support critical applications such as healthcare, industrial automation, and smart cities, where performance, reliability, availability and security in general are paramount [5].

The domain of IoT security involves issues related to both IoT service models and IoT architectures. A simple configuration error in this environment may have serious consequences, since IoT configurations may be shared by many services. There are also other security issues associated with this category; for instance, if an application programming interface (API) covers the vulnerabilities of all IoT APIs, these then become threats and can be exploited by attacks such as man-in-the-middle and side channel attacks, data-centric attacks and backdoors in IoT components.

Intrusion detection systems

As described in the previous section, there are numerous types of attacks that can be launched in a IoT environment. IDSs are effective solutions for detecting and resisting these attacks. They include software or hardware components that perform intrusion detection, record any information detected, create alerts, or execute predefined procedures. The level of security of an IoT service can be increased by providing two intrusion detection techniques, the first of which uses signatures (signature-based intrusion detection or misuse detection) and the second behaviors (anomaly-based intrusion detection) [6]. In a signature-based IDS, attacks are recognized by matching the observed data with known attacks, and this technique therefore relies on a knowledge base of the different existing attacks. This principle of intrusion detection is reactive, and is subject to several constraints; for example, the IDS can detect only listed attacks for which it has the fingerprint, and the database

therefore requires frequent updates [7].

An anomaly-based intrusion detection technique is based on a search for abnormal behavior, and anything that deviates from normal conditions triggers an alarm. This type of detection is effective for unknown attacks, but can create a large number of false positives. Some IDSs combine both approaches to achieve better results [8].

There are two types of IDS in the IoT environment: host-based IDS (HIDS) and network-based IDS (NIDS). A HIDS is responsible for monitoring a computer system, and can be considered as an agent that monitors and analyzes any action, internal or external, that circumvents the system security policy [7]. A NIDS attempts to detect unauthorized access to a network by analyzing the network traffic for signs of malicious activity and events [9].

Limitations of IDSs in the IoT environment

Limitations against unknown attacks

An IDS uses a detection technique based on a database of previously configured attack signatures. Since an unknown attack is unprecedented, no signature can match this threat. An anomaly-based IDS also has difficulty in defining 'normal' behaviors precisely enough to capture subtle variations of an unknown attack without causing a large number of false positives. Existing IDSs lack heuristic intelligence or ML mechanisms that are capable of spotting anomalous behavior or attack patterns that have not been seen before.

Difficulty processing large-scale data

Existing IDSs have a limited capacity for simultaneous data processing, especially in distributed and virtualized platforms such as the IoT, which generate large, complex and diversified data flows. They are often optimized for centralized infrastructures, and have difficulty keeping up with this new scalable architecture, particularly in terms of visibility over all network entry points.

Increasing threat complexity

As attacks evolve, they also become more distributed and complex, and often use advanced methods such as AI, evasion or traffic encryption. These techniques can render an attack invisible to traditional IDSs, which are unable to react to this type of attack due to the massive volumes of data involved.

Theoretical foundations of deep learning for IDSs

In this article, we present an IDS based on ML, which integrates DL techniques. ML is a technique that gives a computer program the ability to acquire skills and interact with complex problems based on data, without being explicitly programmed for each task. These algorithms learn directly from the data, and can improve through experience [10]. ML can be classified into three main categories: supervised learning (SL), unsupervised learning (UL) and reinforcement learning (RL) [11]. SL is based on labeled data, and the algorithm learns to associate inputs (features) with target outputs. This type of learning is often used for classification (prediction of discrete values) and regression (prediction of continuous values) tasks. In UL, unlabeled data are analyzed to discover underlying patterns or structures, while in RL, an agent is taught to interact with its environment by maximizing a cumulative reward [12].

DL is a subset of ML that uses specific algorithmic structures called neural networks, which are modeled on the human brain and include CNNs, recurrent neural networks (RNNs), and deep neural networks (DNNs) [11,13]. These models offer impressive capabilities in regard to automatic hierarchical feature extraction, and can enable the processing of raw data, temporal sequences and complex attributes. Unlike traditional IDS approaches that rely on fixed rules or manually defined features, DL models can automatically learn complex patterns, thus enhancing the detection of anomalies and sophisticated cyberattacks.

Table 1

Attributes of the CICIDS2017 dataset.

No.	Feature	No.	Feature	No.	Feature
1	Flow ID	29	Fwd IAT Std	57	ECE Flag Count
2	Source IP	30	Fwd IAT Max	58	Down/Up Ratio
3	Source Port	31	Fwd IAT Min	59	Average Packet Size
4	Destination IP	32	Bwd IAT Total	60	Avg Fwd Segment Size
5	Destination Port	33	Bwd IAT Mean	61	Avg Bwd Segment Size
6	Protocol	34	Bwd IAT Std	62	Fwd Avg Bytes/Bulk
7	Timestamp	35	Bwd IAT Max	63	Fwd Avg Packets/Bulk
8	Flow Duration	36	Bwd IAT Min	64	Fwd Avg Bulk Rate
9	Total Fwd Packets	37	Fwd PSH Flags	65	Bwd Avg Bytes/Bulk
10	Total Backward Packets	38	Bwd PSH Flags	66	Bwd Avg Packets/Bulk
11	Total Len. of Fwd Packets	39	Fwd URG Flags	67	Bwd Avg Bulk Rate
12	Total Len. of Bwd Packets	40	Bwd URG Flags	68	Subflow Fwd Packets
13	Fwd Packet Length Max	41	Fwd Header Length	69	Subflow Fwd Bytes
14	Fwd Packet Length Min	42	Bwd Header Length	70	Subflow Bwd Packets
15	Fwd Packet Length Mean	43	Fwd Packets/s	71	Subflow Bwd Bytes
16	Fwd Packet Length Std	44	Bwd Packets/s	72	Init_Win_bytes_fwd
17	Bwd Packet Length Max	45	Min Packet Length	73	Act_data_pkt_fwd
18	Bwd Packet Length Min	46	Max Packet Length	74	Min_seg_size_fwd
19	Bwd Packet Length Mean	47	Packet Length Mean	75	Active Mean
20	Bwd Packet Length Std	48	Packet Length Std	76	Active Std
21	Flow Bytes/s	49	Packet Len. Variance	77	Active Max
22	Flow Packets/s	50	FIN Flag Count	78	Active Min
23	Flow IAT Mean	51	SYN Flag Count	79	Idle Mean
24	Flow IAT Std	52	RST Flag Count	80	Idle Packet
25	Flow IAT Max	53	PSH Flag Count	81	Idle Std
26	Flow IAT Min	54	ACK Flag Count	82	Idle Max
27	Fwd IAT Total	55	URG Flag Count	83	Idle Min
28	Fwd IAT Mean	56	CWE Flag Count	84	Label

In this research, we propose a supervised IDS (using SL) based on CNNs combined with a feature selection method. A notable advantage of using CNNs in the IDS framework is their ability to capture local dependencies and analyze structural relationships in the data, especially for complex scenarios such as network traffic analysis. This approach improves the efficiency of the CNN model, while enhancing its generalizability.

Related work

Several studies have applied deep learning-based IDSs to secure systems against attacks. However, most studies focus on well-defined networks, making them less suitable for dynamic and distributed IoT environments. DL-based IDSs represent a new paradigm for intrusion detection, already explored by several researchers. In addition, ML and DL techniques have been used with a wide range of evaluation metrics, including accuracy, loss, precision, recall, F1-score, AUC ROC, FPR, and TPR. A comparative review of some of these studies is presented as below, which we regard as a crucial reference point for our proposed strategy.

Hizal et al. [6] proposed a DL-based IDS using the CICIOT2023 dataset to address challenges from IoT device diversity and evolving traffic. After extensive data pre-processing, a two-tier IDS was developed that integrated binary and multiclass classifiers for DDoS attack detection. The authors used neural network models (CNN, DNN and LSTM) and demonstrated the superior performance of their approach over basic models, which offered a robust solution for real-time detection. Using the UNSW-NB15 dataset, Abed et al. [14] applied feature selection methods like PCA and SVD to enhance detection accuracy. The authors evaluated ridge regression (RR), stochastic gradient descent (SGD) and CNN classifiers, and demonstrated significant improvements in accuracy, especially in binary classification. Kasongo [15] designed a framework leveraging RNNs, particularly LSTM architectures, to capture temporal dependencies in network data. To evaluate the effectiveness of the proposed IDS framework, the authors employed the NSL-KDD and UNSW-NB15 benchmark datasets. They further introduced an XGBoost-based feature selection algorithm to reduce the dimensionality

of the feature space for each dataset.

Alzughaihi and Khediri [16] proposed a backpropagation-based DNN trained on the CSE-CICIDS2018 dataset, achieving 98.41 % accuracy in multiclass classification; however, its evaluation on a single dataset limits generalizability. The study also did not account for training and validation times, which could influence its practical applicability for future research. Hnamte et al. [17] proposed an LSTM-AE model for attack detection, and trained and validated it on two public datasets: CICIDS2017 and CSE-CICIDS2018. The model achieved accuracy of 99.10%. However, the training process required significant time and high-end computational resources, making it challenging to apply the model to larger datasets. Yao et al. [18] proposed an unsupervised DL model using bidirectional Generative adversarial networks for anomaly detection in IoT networks. They incorporated the Wasserstein distance into their proposal to improve detection performance, and tested their model on the UNSW-NB15 and CICIDS2017 datasets.

A staged detection model in which network traffic was classified as either normal data or an attack was proposed by Nguyen et al. [2]. Their model first checked traffic using a soft-ordering CNN, and if the traffic was normal, an isolation-based analysis technique and a local outlier-based anomaly detection scheme were used to detect unknown attacks. Zhang et al. [19] proposed a two-phase IDS approach combining ML and DL to analyze IoT network traffic using the CSE-CICIDS2018 dataset. In the first phase, the LightGBM algorithm was used to evaluate and compare six traditional ML algorithms, which classified network traffic as normal or abnormal data. Then, in the second phase, a CNN was applied to samples identified as abnormal to perform an accurate classification of attack types.

Termos et al. [20] proposed GDLC, a graph deep learning framework using a dynamic selection algorithm to identify optimal centrality measures. This process was based on topological network attributes extracted from traffic data, and enabled optimized network analysis and modeling. The authors of [21] introduced IGRF-RFE, a feature selection method for multi-class network anomaly detection using MLP that combined information gain with random forest (RF) to filter out less relevant features and applied recursive feature elimination to refine the selection. The method was tested on the UNSW-NB15 dataset, and was

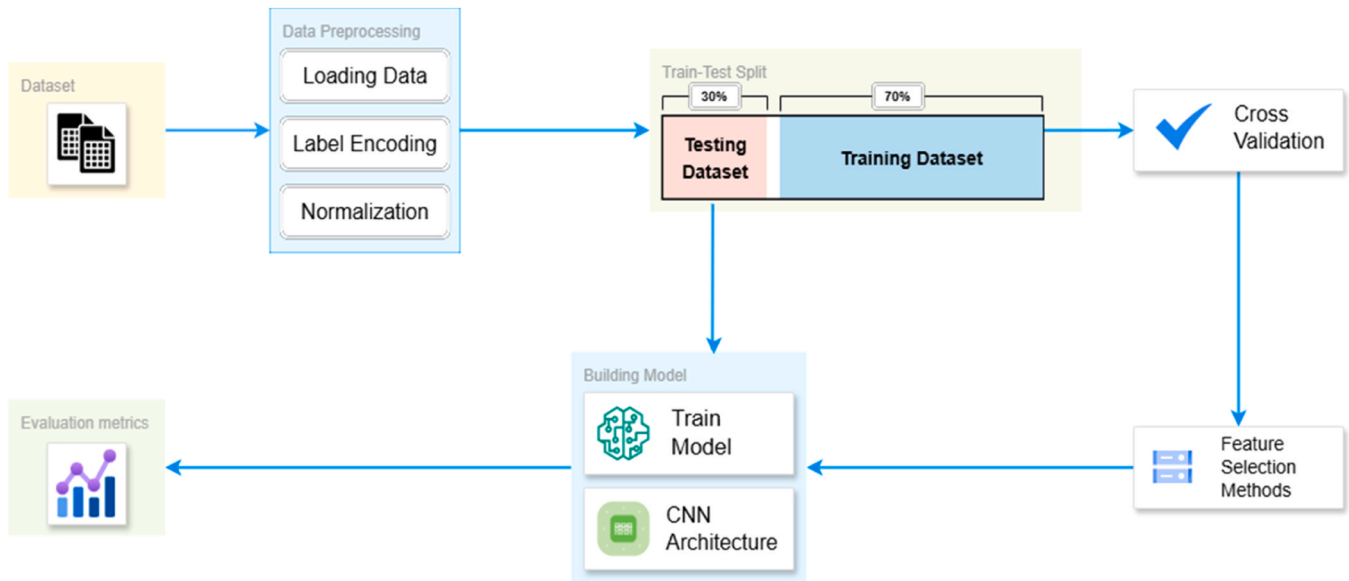


Fig. 1. Proposed CNN-based IDS model.

found to reduce the features from 42 to 23 and to improve the classification accuracy from 82.25 % to 84.24 %, thus demonstrating its ability to enhance detection by focusing on relevant features.

Recent studies have highlighted several critical challenges for IDS. Yee et al. [22] presented a systematic literature review of artificial intelligence methods for detecting zero-day attacks, highlighting the need for models capable of generalizing in the face of unseen threats. Dai et al. [23] proposed integrating autoencoders for anomaly detection with methods such as XGBoost and Random Forest, using the CIC-MalMem-2022 dataset. These hybrid models achieved perfect performance, including on unseen data, highlighting their high effectiveness in detecting zero-day attacks. Sana et al. [24] introduced an approach based on transformer visions to enhance anomaly detection in the IoT, offering a complementary perspective to CNN models. Finally, Yang et al. [25] introduce QB-Auto-IDS, a decentralized IDS based on blockchain technology and hybrid Double Q-Learning with Bi-LSTM, designed for IoT environments. This solution aims to improve intrusion detection in consumer electronics environments by combining robustness, autonomy, and distributed security.

Datasets

Datasets are generally used to evaluate and compare the performance of IDS models based on ML. Each dataset contains network traffic data that can be labeled either as normal traffic, or as various types of attack. These datasets help in benchmarking and evaluating the performance of a CNN-based IDS in the IoT or general network environments.

As reported in the survey published in [11], the majority of proposed IDSs used the KDD99 reference dataset and its enhanced version, NSL-KDD: 68 % of proposed models were based on the reference datasets, while only 5 % used real-time data from simulated or real environments. However, current IDS models are not sufficiently reliable and applicable in real-life situations due to dataset limitations.

The dataset used in this work to validate our model is CICIDS2017, which was generated by the Canadian Institute for Cyber Security IDS. It is also used to help to train our model for intrusion detection. It contains

the most recent common network attacks, but it also meets all the criteria of real attacks.

The dataset contains a large number of features, often more than 80 per record, covering many aspects of network connections, including a class label that indicates whether the connection is normal or malicious [26]. Table 1 presents an overview of the attributes found in CICIDS2017 CSV files.

Methodology

This section offers a concise description of the proposed IDS model, which is based on a CNN. CNNs are well-suited to capturing intricate patterns within high-dimensional data, which makes them an excellent choice for detecting anomalies in network traffic. Our approach focuses on utilizing the CNN's ability to automatically learn hierarchical feature representations from raw network data, which can enable a relatively accurate classification of network traffic as either normal or attack data. The development of the proposed model is structured into five key stages: data preprocessing, the training-test split, feature selection, model building, and evaluation, as presented in Fig. 1.

Data preprocessing

The first stage of this research involves data preprocessing, which is important in order to prepare the dataset effectively for analysis. This stage consists of three key steps:

- **Loading data:** The dataset is first loaded into the system, allowing us to examine its structure and to identify any categorical or numerical features requiring further processing. In this case, the CICIDS2017 dataset was stored as CSV file with columns representing various network features, and a target label indicating whether the traffic consisted of normal data or an attack.
- **Label encoding:** This technique allows for the conversion of categorical data into numerical form [27]. In this study, the dataset included categorical features such as the protocol type and service, which are non-numeric data; since the proposed model requires

numerical input, these categorical features were encoded using label encoding.

- **Normalization:** After label encoding, we normalize the data using a min-max scaling technique. This technique adjusts the feature values to a standardized range, often [0,1], to ensure that each feature contributes equally to the model and to prevent bias from features with larger value ranges [28]. The formula used to calculate the scaling value is as follows (Eq. (1)):

$$Z_{scal} = (Z - Z_{min}) / (Z_{max} - Z_{min}) \quad (1)$$

where Z_{min} and Z_{max} represent the maximum and minimum values of each column, respectively, that will be calculated.

The algorithm is described below

Algorithm 1. Min-max Scaling Algorithm

```

1 Input :  $Z(z_1, z_2, \dots, z_j)$   $1 \leq j \leq J$ 
2 Output:  $Z_{scal}(z_1^{scal}, z_2^{scal}, \dots, z_n^{scal})$ 
3 Initialize label encoder as an instance of LabelEncoder
4 Initialize minMaxscaler as an instance of MinMaxscaler
5 for  $k$  in 1 to  $J$  do
6     if ( $z_j$  is a nominal attribute) then
7         Step 1 : Encode the nominal attribute
8         Step 2 : Apply Min – Max scaling to the encoded attribute
9         
$$z_j^{scal} = \frac{(z_j - z_j^{(min)})}{(z_j^{(max)} - z_j^{(min)})}$$

10    else
11        Step 1 : Apply Min – Max scaling to the numerical attribute
12        
$$z_j^{scal} = \frac{(z_j - z_j^{(min)})}{(z_j^{(max)} - z_j^{(min)})}$$

13    end if
14 end for

```

Training-test split and cross validation

This is a fundamental step in the development of ML models that

involves dividing the dataset into two subsets: one that is used to train the model, and the other for testing its performance [29]. In this research, the dataset was divided into a training set (70 %) and a testing set (30 %). During the model training process, cross-validation was applied to enhance the robustness of the results. This technique involves training the model on multiple data subsets, and allows for reliable performance evaluation and reduces the risk of overfitting [30]. Cross-validation was integrated into the feature selection process to ensure that the selected features generalized well across different data splits, thus further improving model stability and performance.

Feature selection

Feature selection is an essential step in ML that aims to identify the most relevant features from a dataset. By selecting these key features, we can improve the performance of the model, reduce overfitting, and enhance interpretability [31]. This process is especially important in IDSs where large volumes of network data can contain redundant or irrelevant information that may lead to inefficiencies and decreased

model accuracy.

In our study, we employed embedded feature selection techniques based on Random Forest (RF), XGBoost, and LightGBM. These

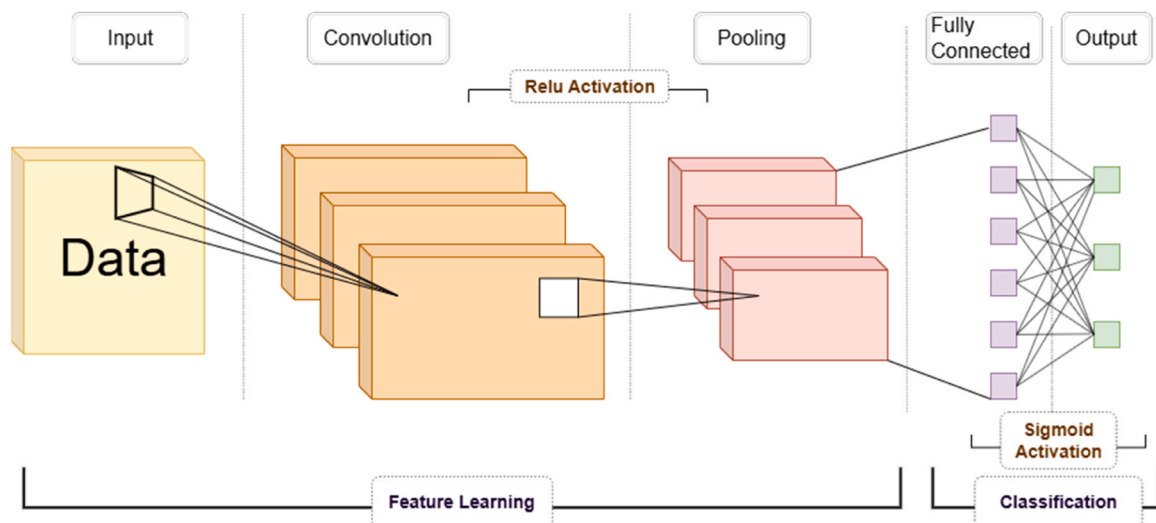


Fig. 2. Proposed CNN architecture.

techniques provide embedded feature selection capabilities, integrating the selection process within model training.

- ❖ **RF:** This technique is based on decision trees, and is mostly used to identify the most relevant features of complex datasets. RF ranks features by importance based on the frequency and effectiveness with which they are used for decision splits across the ensemble of trees [32].
- ❖ **XGBoost:** This is a gradient boosting algorithm based on decision trees that performs exceptionally well in classification and regression tasks [33]. In addition to being a powerful algorithm, XGBoost provides a way to identify the most relevant features in a dataset, making it valuable for feature selection. XGBoost builds a series of decision trees using the boosting principle, where each new tree attempts to correct the errors of the previous trees. During the tree construction process, certain features play a more significant role than others in the splitting decisions, allowing us to quantify their importance.
- ❖ **LightGBM:** This is a type of gradient boosting algorithms that can be used for feature selection. LightGBM provides built-in feature importance scores, which quantify the contribution of each feature to the model [34]. These scores are calculated by evaluating how much each feature helps reduce the error when used in decision tree splits.

Model building

In this section, we design the CNN architecture before implementing and training the proposed model. The architecture is presented in Fig. 2, and is based on a kind of DL model that excels in tasks involving spatial data, by automatically learning features through layers.

In our proposed model, the CNN architecture contains the following components:

- ❖ **Input layer:** This layer represents the initial data fed to the CNN, and serves as the foundation for subsequent layers [35]. The goal of this layer is to structure the data in a way that enables the CNN's convolutional layers to identify and learn patterns that may indicate normal behavior or various forms of network intrusions. In this study, after preprocessing and feature selection, the set of features in the IDS were arranged in a 1D vector format, where each entry in the vector corresponded to one feature of the network data.
- ❖ **Convolutional layer:** The convolutional layer applies a set of learnable filters (kernels) to the input data to extract features. In our case, the data were represented as a 1D vector of network features, and the convolutional layer performed 1D convolution operations [12]. This means it moves a small filter along the input vector, learning specific patterns such as sequences or correlations between features that could indicate intrusion attempts. Each filter detects a particular pattern, such as abnormal packet sequences, unusual session durations, or uncommon protocol use, which could signify suspicious activity. After each convolution operation, an activation function

was applied. The rectified linear unit (ReLU) function was applied after each convolution operation to introduce nonlinearity, which helps the network learn complex patterns. It outputs zero for negative data, and returns the input data unchanged for positive inputs (Eq. (2)).

$$\text{ReLU}(z) = \max(0, z) \quad (2)$$

- ❖ **Pooling layer:** This layer reduces the sizes of the feature maps and focuses on the most important features, which can help the model generalize and reduce the risk of overfitting [36]. There are different types of pooling layer techniques; the most common technique is max pooling, which takes the maximum value from a patch within the feature map. Fig. 3 shows an example of how max pooling works.
- ❖ **Fully connected layer:** After convolution and pooling, the extracted features are passed to fully connected layers to classify network traffic into normal or attack data.
- ❖ **Output layer:** in this study, the problem under consideration is binary classification, so the sigmoid function is used, as shown in Eq. (3). This function outputs a probability score of between zero and one.

$$\text{sigmoid}(t) = \frac{1}{1 + e^{-t}} \quad (3)$$

Model training

Once the architecture has been established, the training process is initiated. The model is trained using labeled data, which allows it to learn the distinguishing features of the network traffic. Parameters such as the learning rate, batch size, and number of epochs are set to optimize performance. For the loss function, we used the binary cross-entropy loss, which calculates the error between the predicted probability of the positive class and the actual class label. The cross-entropy loss is defined as (Eq. (4)):

$$E = -\frac{1}{N} \sum_{i=1}^N (y_i \times \log(\hat{y}_i) + (1 - y_i) \times \log(1 - \hat{y}_i)) \quad (4)$$

where N denotes the number of samples, y_i represents the true label for each sample (zero for normal, one for attack), and $\hat{y}_i$ indicates the value predicted by the model.

Evaluation

The performance of our proposed model, with and without feature selection, was evaluated using several metrics, including accuracy, precision, recall, and F1-score. A confusion matrix was employed to analyze the learning effectiveness of the model, with its components comprising true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) [37]. These metrics offer a comprehensive assessment of the model's ability to distinguish between normal and attack traffic within the network [38].

The accuracy is the ratio of correct predictions to all predictions, as shown in Eq. (5):

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (5)$$

The precision is the ratio of true positives to the sum of false positives and true negatives, as shown in Eq. (6):

$$\text{Precision} = \frac{TP}{TP + FN} \quad (6)$$

The recall (sensitivity) is the ratio of correctly predicted positive observations to all actual positive observations, as given in Eq. (7):

$$\text{Recall} = \frac{TP}{TP + FP} \quad (7)$$

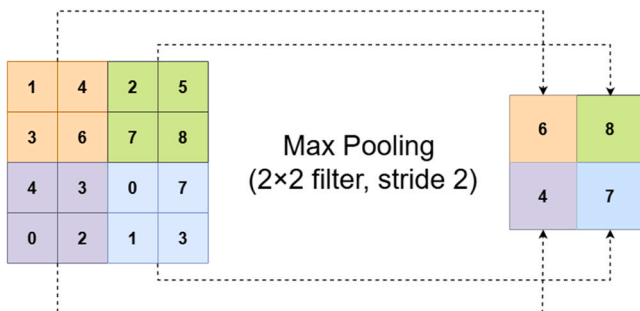


Fig. 3. Example of Max Pooling Process in a CNN architecture.

The F1-score, which is defined as the harmonic mean of precision and recall. It balances the two metrics as shown in Eq. (8):

$$F1 = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \quad (8)$$

The last one is AUC, which in this work specifically corresponds to the AUROC (Area Under the Receiver Operating Characteristic curve) is a performance metric for binary classifiers. It represents the area under the ROC curve, which plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at different classification. The mathematical formula of AUROC is represented by:

$$AUROC = \int_0^1 TPR(FPR) d(FPR)$$

Where TPR, FPR are True Positive Rate and False Positive Rate respectively:

$$TPR = \frac{TP}{TP + FN} \quad (9)$$

$$FPR = \frac{FP}{FP + TN} \quad (10)$$

Experimental results and discussion

Implementation platform and simulation

To implement our proposed model, we used the Google Compute Engine Cloud (Colab) with Python 3 and GPU support, as this offered access to intensive computing resources suitable for training large-scale deep learning models. In particular, the allocated resources included 12.7 GB system RAM, 15 GB GPU VRAM (Tesla T4), and 112.6 GB disk space, ensuring sufficient capacity for deep learning training.

On the software side, we used Python as our programming language with the Keras and TensorFlow libraries to create and train our CNN model. For pre-processing and normalization, we employed scikit-learn. We also used Panda and NumPy for data management and manipulation, and Matplotlib and Seaborn for data visualization.

We first describe the implementation process of our model, in a step-by-step manner, from the data pre-processing phase through to the results analysis and comparison phase.

– Data pre-processing

The first step was to pre-process the CICIDS2017 dataset to ensure quality and model stability, and to prepare it for feature selection. In particular, this involved dealing with missing values, normalizing numerical features using the MinMaxScaler function (in the algorithm presented above), which is used to encode the target variable (Label) and to bring the value to within the interval [0, 1]. This normalization reduces the impact of extreme values and helps our model learn more efficiently.

Table 2

List of Hyperparameters for our proposed model.

Hyperparameter	Value / Configuration
Numbers of filters in convolution layers	32, 16
Activation functions in convolution layers	ReLU
Activation function in output layer	Sigmoid
Loss function	binary_crossentropy for binary classification
Optimizer	Adam
Learning rate	Adaptive
Batch size	32
Number of epochs	30
learning_rate	0.001 (adaptive),
Number of features selected	10, 15, 20, 30, 40

– Balancing the CICIDS2017 dataset with SMOTE and class weights techniques

The CICIDS2017 dataset used in this study has a significant imbalance between classes, with a majority of samples representing only one type of network traffic. In order to make the dataset balanced and to enable more efficient learning, we applied SMOTE to generate synthetic examples [39]. This technique was applied before dividing the dataset into the training and test sets. Moreover, after splitting the dataset, we have applied the class weights technique to balance the model training process. And avoid common biases associated with the dataset and to improve the model's ability to detect different classes of network traffic.

– Training of the CNN model

After balancing the dataset using SMOTE, we trained our model in two stages (without and with feature selection) on the binary classification task, using the hyperparameters presented in Table 2.

The CNN architecture incorporates two convolutional layers, featuring 32 and 16 filters respectively, to ensure adequate representational capability while keeping model complexity and inference time in check. ReLU activations are applied in the convolutional layers to enhance convergence speed, while a sigmoid activation function is utilized at the output for estimating binary probabilities. The network is trained using the Adam optimizer (with an initial learning rate of 0.001) and binary cross-entropy loss, employing a batch size of 32 and a maximum of 30 epochs with early stopping implemented. These configurations align with established best practices and have been empirically validated. We assessed the model across various feature budgets (10, 15, 20, 30, and 40 selected features) to showcase its robustness and to pinpoint the optimal balance between performance and computational expense; the final configuration reported (40 features) achieves the most favorable equilibrium between detection performance and a brief execution time (22 s). Fig. 4 shows architectural details such as the types of layers, numbers of trainable parameters and numbers of units per layer of the proposed CNN model for dataset evaluation.

Results and discussion

Model without feature selection

In the initial phase, simulations were carried out without applying any feature selection (FS) techniques. The values of the model performance metrics following the training process are summarized in Table 3. Fig. 5 illustrates the progression of the model accuracy and loss across the training epochs, providing insights into the model's convergence and stability.

Without the FS technique, the proposed model processes all variables in the dataset, which can introduce unnecessary noise or redundant features. This often leads to lower performance, particularly in terms of accuracy and F1-score.

Model with feature selection

In the second phase of our experimental process, we applied the cross-validation technique, which was integrated into the FS process to ensure that the selected features have an interesting weight, thus further improving the stability and performance of the proposed model. To achieve this, we evaluated FS techniques to identify the best option in terms of the efficiency and performance of our model. We used a comprehensive approach based on CNN involving three commonly used techniques: RF, XGBoost and LightGBM. For the filtering methods, we generated subsets of 10, 15, 20, 30, and 40 features. We then trained the CNN-based IDS model to evaluate the performance of these feature subsets. Throughout this process, we documented all of the results, including the evaluation metrics for each model and the corresponding number of features selected. These metrics enabled us to evaluate the effectiveness of the FS approaches and to find the subsets that produced

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 8, 32)	128
max_pooling1d (MaxPooling1D)	(None, 4, 32)	0
conv1d_1 (Conv1D)	(None, 2, 16)	1,552
max_pooling1d_1 (MaxPooling1D)	(None, 1, 16)	0
dropout (Dropout)	(None, 1, 16)	0
flatten (Flatten)	(None, 16)	0
dense (Dense)	(None, 64)	1,088
dense_1 (Dense)	(None, 1)	65

Fig. 4. Summary of parameters training with CNN architecture.

Table 3
Model performance without FS applied.

Model	Epoch	Accuracy	Precision	Recall	F1-Score	Loss Rate
Proposed CNN without FS	30	95.50	91.85	99.83	95.67	0.0450

the best results.

Table 4 presents performance of CNN model with XGBoost based FS, using different numbers of selected features (10, 15, 20, 30, and 40) evaluated using 5-fold cross-validation. The results show a clear trend: as the number of selected features increases, the performance of the model consistently improves across all metrics with. With only 10 features, the model already achieves a strong accuracy of 99.23 % and an F1-score of 99.22 %. As more features are added, accuracy and F1-score

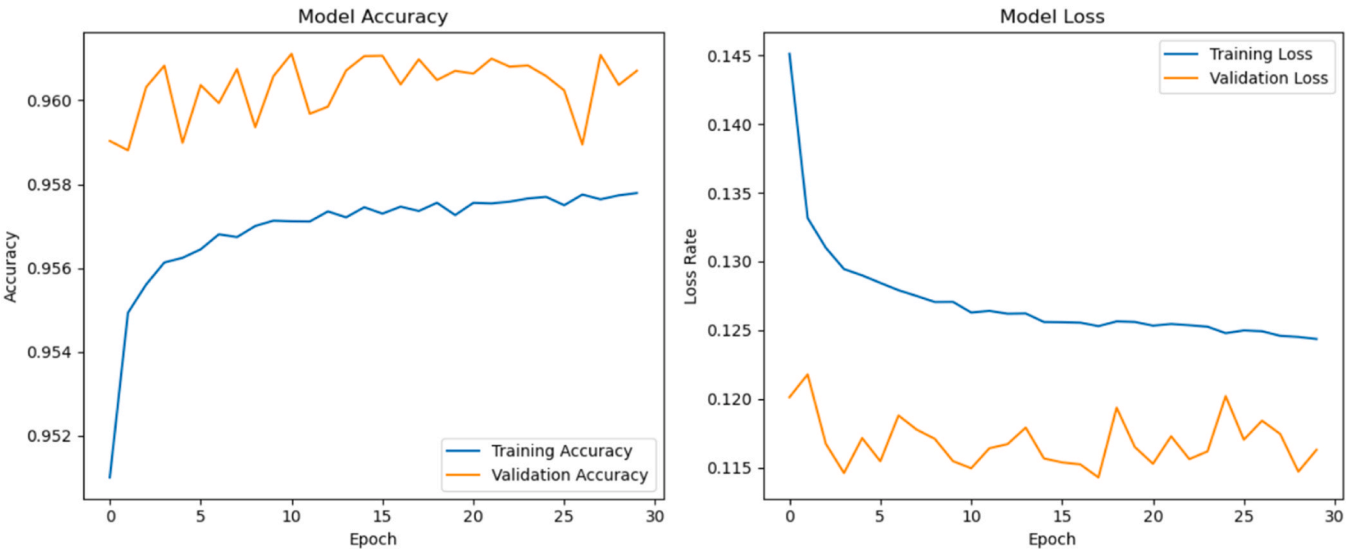


Fig. 5. The progression of our model accuracy and loss across the training epochs without FS.

Table 4
Results of the proposed model combined with XGBoost.

Model	No. FS	K-fold cross validation	Accuracy	Precision	Recall	F1-Score	Loss Rate
CNN + XGBoost	10	K=5	99.23%	99.82%	98.63%	99.22%	0.004
	15		99.85%	100.00%	99.70%	99.85%	0.0015
	20		99.89%	100.00%	99.79%	99.89%	0.0011
	30		99.93%	100%	99.85%	99.93%	0.0007
	40		99.95%	99.99%	99.91%	99.95%	0.0005

Table 5
Performance of CNN Model with random forest based FS.

Model	No. FS	K-fold cross validation	Accuracy	Precision	Recall	F1-Score	Loss Rate
CNN + RF	10	K=5	99.66%	99.60%	99.70%	99.66%	0.0043
	15		99.81%	99.99%	99.63%	99.81%	0.0019
	20		99.84%	100%	99.68%	99.84%	0.0016
	30		99.94%	99.97%	99.90%	99.94%	0.0006
	40		99.95%	99.99%	99.90%	99.95%	0.0005

Table 6
Performance of CNN Model with LightGBM based FS.

Model	No. FS	K-fold cross validation	Accuracy	Precision	Recall	F1-Score	Loss Rate
CNN + LightGBM	10	K=5	99.39%	98.96%	99.83%	99.40%	0.0061
	15		99.75%	99.96%	99.54%	99.75%	0.0025
	20		99.93%	99.96%	99.90%	99.93%	0.0007
	30		99.93%	99.98%	99.87%	99.93%	0.0007
	40		99.95%	100.00%	99.90%	99.95%	0.0005

steadily increase, reaching their highest values at 40 features with 99.95 % accuracy, 99.95 % F1-score, and a very low loss rate of 0.0005.

Table 5 reports the performance of the CNN model when FS was performed using Random Forest (RF) with varying numbers of selected features (10, 15, 20, 30, and 40) evaluated using 5-fold cross-validation. Even with only 10 features, the model demonstrates strong results, achieving 99.66 % accuracy and an F1-score of 99.66 %. As the number of features increases, performance improves consistently, with accuracy and F1-score exceeding 99.8 % from 15 features onward. The best results are obtained with 40 features, where the CNN reaches 99.95 % accuracy, 99.95 % F1-score, and a very low loss rate of 0.0005.

Table 6 shows the performance of the CNN model when combined with LightGBM for FS, using different subsets of selected features (10,

15, 20, 30, and 40) and evaluated using 5-fold cross-validation. With 10 features, the model achieves 99.39 % accuracy and an F1-score of 99.40 %, which already reflects strong performance despite the small feature set. As the number of features increases, performance improves significantly, surpassing 99.7 % accuracy and F1-score at 15 features. The best results are obtained with 40 features, where the CNN reaches 99.95 % accuracy, an F1-score of 99.95 %, and a perfect precision, with the lowest loss rate of 0.0005.

The results show a significant improvement in overall model performance using FS technique. Furthermore, if we compare the values in Tables 3–6 (with and without FS), we see that the metrics increase slightly, from 95.5 % to 99.23 % (XGBoost with 10 features) suggesting that FS helps the model to better discriminate between classes without

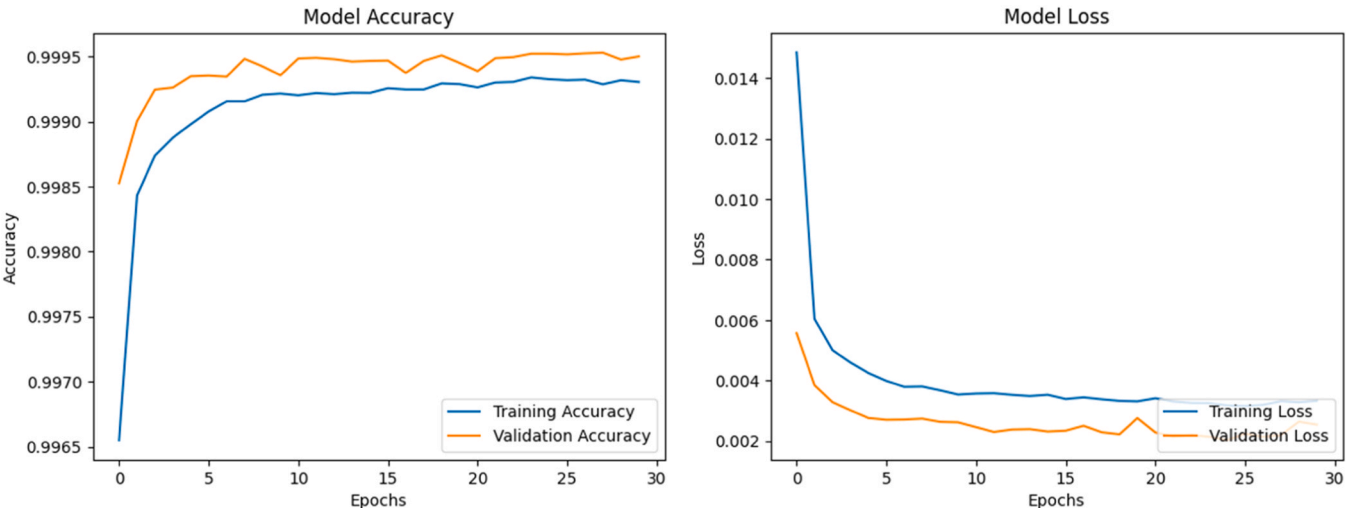


Fig. 6. Evolution of Training and Validation Loss Compared to XGBoost (40 Features).

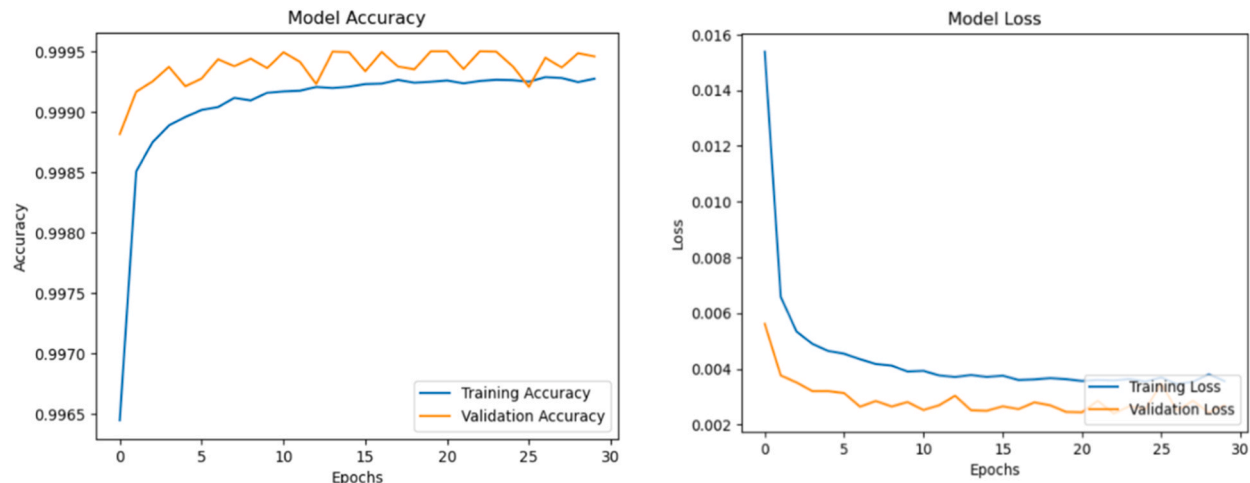


Fig. 7. Evolution of Training and Validation Loss of RF technique with 40 Features.

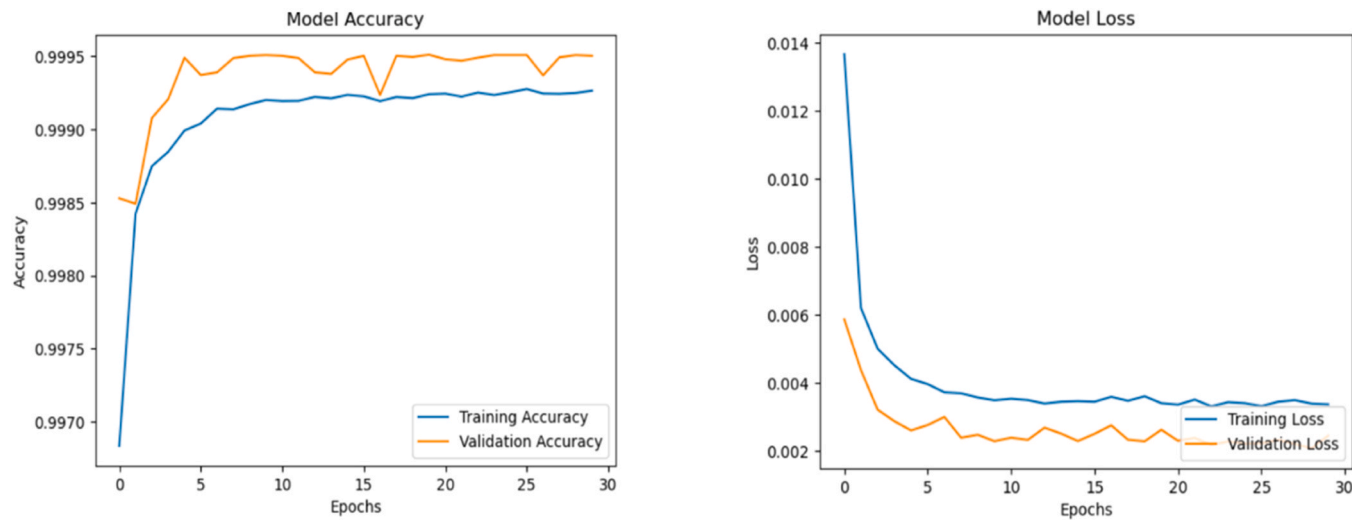


Fig. 8. Evolution of Training and Validation Loss of LightGBM method with 40 Features.

Table 7
Comparative analysis of CNN models combined with different feature selection methods.

Model	No. FS	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	Loss	Run time (s)
CNN+XGBoost	40	99.95 ± 0.001	99.99 ± 0.001	99.90 ± 0.003	99.95 ± 0.001	0.0005±0.0	22
CNN+RF	40	99.93 ± 0.013	99.99 ± 0.006	99.87 ± 0.024	99.93 ± 0.013	0.0007 ± 0.0001	24
CNN+LightGBM	40	99.93 ± 0.004	99.99 ± 0.003	99.87 ± 0.010	99.93 ± 0.004	0.0007± 0.00007	23

increasing unnecessary noise in the data. In addition, the recall score indicates an enhanced ability to distinguish samples from different classes, thereby reinforcing the model’s reliability in terms of intrusion detection.

To assess the performance of the proposed model, we analyze our experimental results to identify the best FS method combined with the proposed CNN architecture, and determine the most effective overall method for classification.

Analysis of results for each selection method

XGBoost. Using 40 features, XGBoost demonstrated excellent performance in term of precision, recall and F1-Score while reducing the loss to just 0.0005. This outcome demonstrates the algorithm’s ability to efficiently capture patterns with this number of features. Although the

performance was also high with 20 and 30 features, the optimal balance between precision and loss was achieved with 40 features. Fig. 6 shows the evolution of the loss during training and validation compared to XGBoost with 40 features.

Random forest. Compared to the XGBoost technique with 40 features, the RF technique gave better overall performance, with an accuracy and F1-score of 99.95 %, with an extremely low loss (0.0005) as shown in Fig. 7. However, the improvement over 30 features is marginal. It also shows a very low loss (0.0006) and comparable performance.

LightGBM. With 40 features, LightGBM achieved high performance with extremely low loss (0.0005) and reasonable processing time (23 s). Although the performance was also high with 20 and 30 features, the best combination of accuracy, loss and processing time was found for 40

Table 8

The top 40 features selected by XGBoost method.

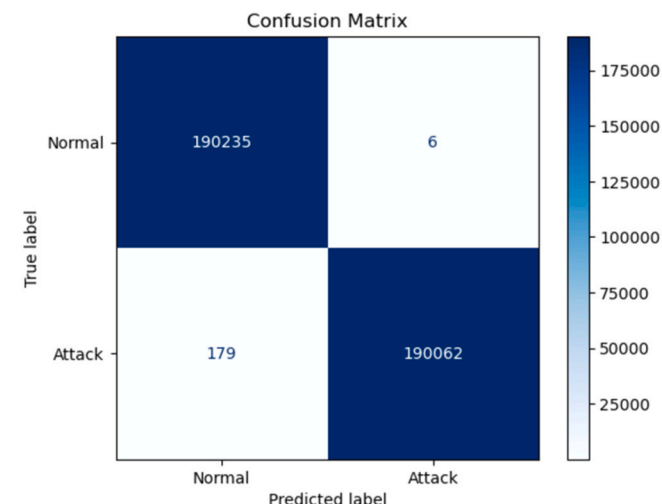
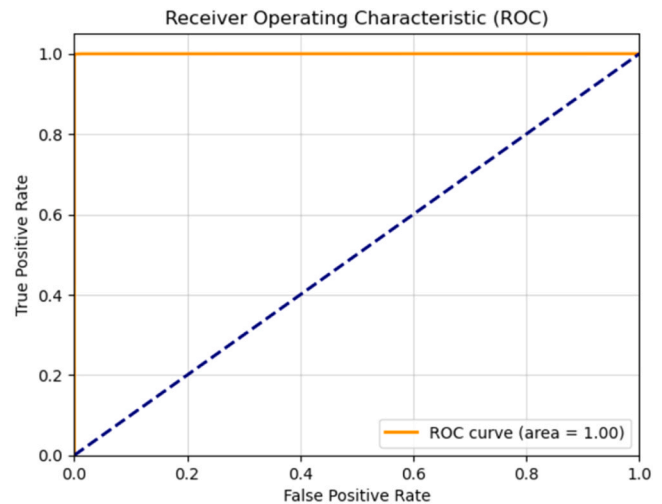
No.	Selected Feature	No.	Selected Feature
1	Total Length of Fwd Packets	21	Flow IAT Std
2	Bwd Packet Length Min	22	Act_data_pkt_fwd
3	Total Length of Bwd Packets	23	Flow IAT Min
4	Min Packet Length	24	ACK Flag Count
5	Flow Bytes/s	25	Flow IAT Max
6	Max Packet Length	26	Bwd Packet Length Mean
7	Packet Length Mean	27	Fwd Packet Length Max
8	PSH Flag Count	28	Fwd Packets/s
9	Flow Duration	29	Total Fwd Packets
10	Init_Win_bytes_backward	30	Fwd IAT Max
11	Bwd Packet Length Std	31	Fwd IAT Total
12	Destination Port	32	Bwd IAT Std
13	Bwd Packets/s	33	Fwd IAT Mean
14	min_seg_size_forward	34	Bwd IAT Min
15	Fwd Header Length	35	Packet Length Std
16	Fwd Packet Length Std	36	Bwd IAT Total
17	Idle Min	37	Flow IAT Mean
18	Bwd Header Length	38	Fwd IAT Std
19	Init_Win_bytes_forward	39	Active Max
20	Fwd IAT Min	40	Idle Std

features. Fig. 8 shows these results over different epochs.

Comparison of global methods. The Table 7 illustrates the comparative analysis of three hybrid models that integrate a CNN with various feature selection methods: XGBoost, RF, and LightGBM. Overall, all three models exhibit exceptionally high overall accuracy, with an average accuracy of 99.95 %, an F1-score of 99.95 %, and an extremely low loss of 0.0005. To evaluate robustness, each configuration was executed multiple times under different random seeds (0, 21, 42, 84, and 123).

Among these, the CNN+XGBoost model, evaluated across multiple runs, shows an average accuracy of 99.95 % with a standard deviation of ± 0.001 and a minimal loss of 0.0005 ± 0.0 , indicating remarkable performance stability. Similarly, the corresponding precision (99.99 ± 0.001) and recall (99.90 ± 0.003) indicate a well-balanced detection rate with negligible false positives.

In comparison, the CNN+RF and CNN+LightGBM models achieve comparable results, with identical accuracy (99.93 %) and F1-scores (99.87) with standard deviation ≤ 0.013 , confirming consistent yet slightly less stable performance. The execution time varies slightly among the models: 22 s for CNN+XGBoost, 23 s for CNN+LightGBM, and 24 s for CNN+RF, indicating that all three are efficient and comparable in terms of speed. In summary, these results not only

**Fig. 9.** Confusion matrix from the XGBoost model.**Fig. 10.** ROC curve from the XGBoost model.

demonstrate the reliability of CNN models combined with advanced feature selection methods but also the stability of their performance, particularly when standard deviations are low, as observed for CNN+XGBoost.

For greater clarity and reproducibility of our proposal, Table 8 illustrates the top 40 features selected by the XGBoost method. These attributes include indicators related to packets (e.g., Total Length of Fwd Packets, Bwd Packet Length Min), flow statistics (Flow Duration, Flow Bytes/s, IAT), and some protocol variables (PSH Flag Count, ACK Flag Count, etc.).

XGBoost with 40 features was found to be the best-performing model, as it achieved perfect accuracy, a low loss, and a reasonable runtime. The results demonstrate that combining advanced FS techniques (such as XGBoost or LightGBM) with DL-based models (CNN) can offer optimized solutions for an IDS in the context of IoT. The confusion matrix, presented in Fig. 9, demonstrates the excellent performance of our XGBoost model for binary classification between “normal” and “attack” classes, with only 179 false negatives and 6 false positives out of approximately 380,000 samples. This demonstrates the reliability of our model in minimizing false alarms while ensuring accurate detection of attacks.

Fig. 10 shows the ROC curve, where the area under the curve (AUC) reaches 1.00, indicating perfect discrimination ability. To complement this, we also generated the precision-recall (PR) curve (fig), which further highlights the excellent detection performance, with values consistently close to 1.0 despite the inherent imbalance in the CICIDS2017 dataset.

Finally, a brief analysis of errors reveals that the majority of classification errors occur in cases where some benign traffic shares features similar to those of malicious traffic (false positives) and in rare cases of highly obfuscated attacks (false negatives). This information confirms the robustness of the proposed model while suggesting possible areas for improvement in feature engineering in future work.

Discussion and comparison

In this section, we present a comparison of the experimental results obtained using our model with FS techniques (RF, LightGBM and XGBoost) with similar studies in the literature. This comparison allows us to validate the performance and assess the effectiveness of our contributions in relation to existing research.

The Table 9 presents a comparison of our results with those of existing schemes, and it can be seen that the proposed model shows promising results compared to the other models. In particular, the proposed model achieved the best performance in terms of accuracy,

Table 9
A comparison of the proposed model to the state-of-the-art on CICIDS2017 dataset.

Paper	Year	Model	Dataset	No. FS	Accuracy	Precision	Recall	F1 Score
[40]	2023	CNN	CICIDS2017	71	99.61 %	97.05 %	95.00 %	93.01 %
[41]	2023	LSTM-RNN	CICIDS2017	75	99.85 %	100 %	99.70 %	99.85 %
[42]	2023	DNN	CICIDS2017	64	99.80 %	99.85 %	99.94 %	99.89 %
[43]	2023	LSTM	CICIDS2017	-	99.83 %	97.95 %	99.79 %	98.96 %
[35]	2023	DNN	CICIDS2017	-	99.80 %	99.80 %	99.80 %	99.80 %
[44]	2024	AE+PCA+LSTM	CICIDS2017	-	93.78 %	96.57 %	95.65 %	96.11 %
[45]	2024	EDA-Xgboost	CICIDS2017	-	99.87 %	99.75 %	99.99 %	99.87 %
Our proposed Model	2025	CNN-Xgboost	CICIDS2017	40	99.95 %	99.99 %	99.91 %	99.95 %

precision, recall and F1-score, by leveraging FS technique; this finding is justified by the confusion matrix and the ROC curve, which show a remarkable ability to detect anomalies with an extremely low false positive and false negative rates. This finding is supported by the work in [15], whose authors used XGBoost with an elastic deep autoencoder (EDA) and achieved results close to our own (accuracy 99.87 %). Although the table does not provide execution times for all models, our model stands out for its remarkably short execution time of 22 s. This makes it ideal for real-time modeling, and it outperforms comparable models reported in existing works.

Conclusion and perspectives

This paper proposes an efficient IDS based on CNN combined with FS (XGBoost, LightGBM, RF) for binary classification (normal vs. attack). The aim was to enhance intrusion detection performance in IoT environments. According to the experimental results, our CNN-based IDS model demonstrated significant improvements in accuracy (99.95 %) and precision (99.99 %), especially with the XGBoost technique, while offering optimal results in terms of execution time (22 s).

As a future perspective, and in order to enhance and approved our proposed model in real time environment, we will deploy and assess our approach on lightweight devices (e.g., Raspberry Pi, Jetson Nano). Furthermore, optimization techniques such as pruning and quantization were implemented. To better reflect the diversity of network threats, we aim to extend our model to multiclass classification, exploring hybrid deep learning architectures such as CNN combined with LSTM, which are better suited for capturing both spatial and temporal features in traffic data.

CRedit authorship contribution statement

EL KIRAM My Ahmed: Supervision, Investigation, Funding acquisition. **Outmane Bourkhoukou:** Visualization, Investigation, Data curation. **Toufik Datsi:** Software, Formal analysis, Data curation. **Omar Achbarou:** Writing – review & editing, Writing – original draft, Validation, Supervision, Methodology, Investigation.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] J. Yang, et al., DDoS attack detection in consumer IoT-based healthcare systems using improved off-policy proximal policy optimization and generative adversarial network, *IEEE Trans. Consum. Electron.* (2025) 1, <https://doi.org/10.1109/TCE.2025.3605098>.
[2] X.H. Nguyen, K.H. Le, Robust detection of unknown DoS/DDoS attacks in IoT networks using a hybrid learning model, *Internet Things (Neth.)* 23 (2023), <https://doi.org/10.1016/j.iot.2023.100851>.
[3] S. Bamohabbat Chafjiri, P. Legg, J. Hong, M.A. Tsompanas, Vulnerability detection through machine learning-based fuzzing: A systematic review, *Comput. Secur* 143 (2024), <https://doi.org/10.1016/j.cose.2024.103903>.

[4] L. Chen, X. Kuang, A. Xu, S. Suo, Y. Yang, A Novel Network Intrusion Detection System Based on CNN, 2020 Eighth International Conference on Advanced Cloud and Big Data (CBD) (2020) 243–247, <https://doi.org/10.1109/CBD51900.2020.00051>.
[5] E. Rios, M. Higuero, X. Larrucea, M. Rak, V. Casola, E. Iturbe, Security and Privacy Service Level Agreement composition for Internet of Things systems on top of standard controls, *Comput. Electr. Eng.* 98 (2022) 107690, <https://doi.org/10.1016/j.compeleceng.2022.107690>.
[6] S. Hizal, U. Cavusoglu, D. Akgun, A novel deep learning-based intrusion detection system for IoT DDoS security, *Internet Things* 28 (2024) 101336, <https://doi.org/10.1016/j.iot.2024.101336>.
[7] S. Li, Y. Cao, S. Liu, Y. Lai, Y. Zhu, N. Ahmad, HDA-IDS: a hybrid DoS attacks intrusion detection system for IoT by using semi-supervised CL-GAN, *Expert Syst. Appl.* 238 (2024) 122198, <https://doi.org/10.1016/j.eswa.2023.122198>.
[8] O. Achbarou, A new distributed intrusion detection system based on multi-agent system for cloud environment, no. 3, *IJCNIS* 10 (2018), (<http://www.ijcnis.org/index.php/ijcnis/article/view/3546>) [Online]. Available.
[9] S.Z. Mehdi, Syed Ali, Hussain, Survey on intrusion detection system in IoT network, in: A. B. S., H. A. E., A. S., J.A. Gupta Deepak, Khanna (Eds.), *International Conference on Innovative Computing and Communications*, Springer Nature Singapore, Singapore, 2023, pp. 721–732.
[10] A.S. Dina, D. Manivannan, Intrusion detection based on machine learning techniques in computer networks, *Internet Things* 16 (2021) 100462, <https://doi.org/10.1016/j.iot.2021.100462>.
[11] A. Aldweesh, A. Derhab, A.Z. Emam, Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues, *Knowl. Based Syst.* 189 (2020) 105124, <https://doi.org/10.1016/j.knsys.2019.105124>.
[12] T. Bellam, P. Lakshmi Prasanna, Novel DeepLearning based sentimental approach to identifying the fake news in social networking media based smart application, *Meas. Sens.* 33 (Jun. 2024) 101148, <https://doi.org/10.1016/J.MEASEN.2024.101148>.
[13] B. Lampe, W. Meng, A survey of deep learning-based intrusion detection in automotive applications, *Expert Syst. Appl.* 221 (2023) 119771, <https://doi.org/10.1016/j.eswa.2023.119771>.
[14] R.A. Abed, E.K. Hamza, A.J. Humaidi, A modified CNN-IDS model for enhancing the efficacy of intrusion detection system, *Meas. Sens.* 35 (2024) 101299, <https://doi.org/10.1016/J.MEASEN.2024.101299>.
[15] S.M. Kasongo, A deep learning technique for intrusion detection system using a Recurrent Neural Networks based framework, *Comput. Commun.* 199 (2023) 113–125, <https://doi.org/10.1016/J.COMCOM.2022.12.010>.
[16] S. Alzughaibi, S. El Khediri, A cloud intrusion detection systems based on DNN using backpropagation and PSO on the CSE-CIC-IDS2018 dataset, *Appl. Sci.* 13 (4) (2023), <https://doi.org/10.3390/app13042276>.
[17] V. Hnamte, H. Nhung-Nguyen, J. Hussain, Y. Hwa-Kim, A novel two-stage deep learning model for network intrusion detection: LSTM-AE, *IEEE Access* 11 (2023) 37131–37148, <https://doi.org/10.1109/ACCESS.2023.3266979>.
[18] W. Yao, H. Shi, H. Zhao, Scalable anomaly-based intrusion detection for secure Internet of Things using generative adversarial networks in fog environment, *J. Netw. Comput. Appl.* 214 (2023), <https://doi.org/10.1016/j.jnca.2023.103622>.
[19] H. Zhang, B. Zhang, L. Huang, Z. Zhang, H. Huang, An efficient two-stage network intrusion detection system in the internet of things, *Inf. (Switz.)* 14 (2) (2023), <https://doi.org/10.3390/info14020077>.
[20] M. Termos, Z. Ghalmane, M. el, A. Brahmia, A. Fadlallah, A. Jaber, M. Zghal, GDLC: A new Graph Deep Learning framework based on centrality measures for intrusion detection in IoT networks, *Internet Things (Neth.)* 26 (2024), <https://doi.org/10.1016/j.iot.2024.101214>.
[21] Y. Yin, et al., IGRF-RFE: a hybrid feature selection method for MLP-based network intrusion detection on UNSW-NB15 dataset, *J. Big Data* 10 (1) (2023), <https://doi.org/10.1186/s40537-023-00694-8>.
[22] L. Yee Por, et al., A systematic literature review on AI-based methods and challenges in detecting zero-day attacks, *IEEE Access* 12 (2024) 144150–144163, <https://doi.org/10.1109/ACCESS.2024.3455410>.
[23] Z. Dai, et al., An intrusion detection model to detect zero-day attacks in unseen data using machine learning, *PLoS One* 19 (9) (2024) e0308469, <https://doi.org/10.1371/journal.pone.0308469>.
[24] L. Sana, et al., Securing the IoT cyber environment: enhancing intrusion anomaly detection with vision transformers, *IEEE Access* 12 (2024) 82443–82468, <https://doi.org/10.1109/ACCESS.2024.3404778>.
[25] J. Yang, et al., QB-AutoIDS: a blockchain-based decentralized autonomous cyberattack detection system for consumer electronics using hybrid double Q-

- learning and Bi-LSTM, *IEEE Trans. Consum. Electron.* (2025) 1, <https://doi.org/10.1109/TCE.2025.3600429>.
- [26] J. Liu, Y. Gao, F. Hu, A fast network intrusion detection system using adaptive synthetic oversampling and LightGBM, *Comput. Secur.* 106 (2021) 102289, <https://doi.org/10.1016/j.cose.2021.102289>.
- [27] J.T. Hancock, T.M. Khoshgoftaar, Survey on categorical data for neural networks, *J. Big Data* 7 (1) (2020) 28, <https://doi.org/10.1186/s40537-020-00305-w>.
- [28] L. Huang, J. Qin, Y. Zhou, F. Zhu, L. Liu, L. Shao, Normalization techniques in training DNNs: methodology, analysis and application, *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (8) (2023) 10173–10196, <https://doi.org/10.1109/TPAMI.2023.3250241>.
- [29] J.J. Salazar, L. Garland, J. Ochoa, M.J. Pyrcz, Fair train-test split in machine learning: Mitigating spatial autocorrelation for improved prediction accuracy, *J. Pet. Sci. Eng.* 209 (2022) 109885, <https://doi.org/10.1016/j.petrol.2021.109885>.
- [30] A.A. Khan, O. Chaudhari, R. Chandra, A review of ensemble learning and data augmentation models for class imbalanced problems: combination, implementation and evaluation, *Expert Syst. Appl.* 244 (2024) 122778, <https://doi.org/10.1016/j.eswa.2023.122778>.
- [31] D. Theng, K.K. Bhojar, Feature selection techniques for machine learning: a survey of more than two decades of research, *Knowl. Inf. Syst.* 66 (3) (2024) 1575–1637, <https://doi.org/10.1007/s10115-023-02010-5>.
- [32] M. Salem, M. EL-Sayed Gabr, M. Mossad, H. Mahanna, Random Forest modelling and evaluation of the performance of a full-scale subsurface constructed wetland plant in Egypt, *Ain Shams Eng. J.* 13 (6) (2022) 101778, <https://doi.org/10.1016/j.asej.2022.101778>.
- [33] A. Ibrahim Ahmed Osman, A. Najah Ahmed, M.F. Chow, Y. Feng Huang, A. El-Shafie, Extreme gradient boosting (Xgboost) model to predict the groundwater levels in Selangor Malaysia, *Ain Shams Eng. J.* 12 (2) (2021) 1545–1556, <https://doi.org/10.1016/j.asej.2020.11.011>.
- [34] W. Chen, H. Yang, L. Yin, X. Luo, Large-scale IoT attack detection scheme based on LightGBM and feature selection using an improved salp swarm algorithm, *Sci. Rep.* 14 (1) (2024) 19165, <https://doi.org/10.1038/s41598-024-69968-2>.
- [35] V. Hnamte, J. Hussain, Dependable intrusion detection system using deep convolutional neural network: A Novel framework and performance evaluation approach, *Telemat. Inform. Rep.* 11 (2023), <https://doi.org/10.1016/j.teler.2023.100077>.
- [36] A. Zafar, et al., A comparison of pooling methods for convolutional neural networks, *Appl. Sci.* 12 (17) (2022), <https://doi.org/10.3390/app12178643>.
- [37] S. Latif, W. Boulila, A. Koubaa, Z. Zou, J. Ahmad, DTL-IDS: an optimized intrusion detection framework using deep transfer learning and genetic algorithm, *J. Netw. Comput. Appl.* 221 (2024) 103784, <https://doi.org/10.1016/j.jnca.2023.103784>.
- [38] A. Pashaei, M.E. Akbari, M. Zolfy Lighvan, A. Charmin, Early intrusion detection system using honeypot for industrial control networks, *Results Eng.* 16 (2022) 100576, <https://doi.org/10.1016/j.rineng.2022.100576>.
- [39] S. Alkadi, S. Al-Ahmadi, M.M. Ben Ismail, Toward improved machine learning-based intrusion detection for internet of things traffic, *Computers* 12 (8) (2023), <https://doi.org/10.3390/computers12080148>.
- [40] J. Jose, D.V. Jose, Deep learning algorithms for intrusion detection systems in internet of things using CIC-IDS 2017 dataset, *Int. J. Electr. Comput. Eng.* 13 (1) (2023) 1134–1141, <https://doi.org/10.11591/ijece.v13i1.pp1134-1141>.
- [41] M. Gamal, M. Elhamahmy, S. Taha, H. Elmahdy, Improving intrusion detection using LSTM-RNN to protect drones' networks, *Egypt. Inform. J.* 27 (2024), <https://doi.org/10.1016/j.eij.2024.100501>.
- [42] A. Thakkar, R. Lohiya, Fusion of statistical importance for feature selection in deep neural network-based intrusion detection system, *Inf. Fusion* 90 (2023) 353–363, <https://doi.org/10.1016/j.inffus.2022.09.026>.
- [43] A. M. Abdulghani, M. M. Abdulghani, W. L. Walters, K. H. Abed, Improving intrusion detection in UAV communication using an LSTM-SMOTE classification method, *J. Cyber Secur.* 4 (4) (2023) 287–298, <https://doi.org/10.32604/jcs.2023.042486>.
- [44] A. Thakkar, N. Kikani, R. Geddam, Fusion of linear and non-linear dimensionality reduction techniques for feature reduction in LSTM-based intrusion detection system, *Appl. Soft Comput.* 154 (Mar. 2024), <https://doi.org/10.1016/j.asoc.2024.111378>.
- [45] D.D. Bikila, J. Čapek, Machine learning-based attack detection for the Internet of Things, *Future Gener. Comput. Syst.* (2024) 107630, <https://doi.org/10.1016/j.future.2024.107630>.